# Project Overview: Campusly Marketplace

**Authoritative Technical Reference & Handover Document**
*Generated based on direct codebase inspection of routes, schemas, controllers, services, guards, and active configurations.*

## 1. Project Summary

Campusly Marketplace is a specialized digital peer-to-peer (P2P) web marketplace engineered specifically for university students and academic communities. The platform enables students to buy, sell, and exchange academic resources, study guides, code repositories, course notes, digital project templates, and freelance technical services within a verified campus ecosystem.

By restricting access to academic networks and introducing a multi-tiered student verification workflow, Campusly addresses the trust and quality deficits common in generic classifieds platforms. Sellers can upload digital deliverable files securely gated behind instant transaction authorization, while buyers gain immediate access to course materials verified by campus peers.

The platform balances peer autonomy with centralized oversight. All user listings undergo an admin moderation queue before public publication, while administrators maintain continuous visibility over platform activity, seller compliance, transactional ledgers, user verification queues, and system-wide performance metrics.

# 2. Tech Stack

All technology dependencies and exact version numbers have been extracted directly from the root `package.json` and backend `node_modules` manifests:

## Frontend

- **Framework**: Next.js (`v16.2.10`) App Router architecture
- **Core Library**: React (`v19.2.4`), React DOM (`v19.2.4`)
- **Styling & Design System**: TailwindCSS (`v4.0.0`) with `@tailwindcss/postcss` (`v4.0.0`), CSS Modules, custom HSL design tokens, glassmorphism, and dark mode styling
- **Animations & Micro-interactions**: Framer Motion (`v12.43.0`)
- **Iconography**: Lucide React (`v1.25.0`)
- **Form Management**: React Hook Form (`v7.82.0`)
- **Date Utilities**: Date-fns (`v4.4.0`)
- **Language & Tooling**: TypeScript (`v5.x`), ESLint (`v9.x`) with `eslint-config-next` (`v16.2.10`)

## Backend API

- **Framework**: NestJS (`v11.1.28`) modular architecture (`@nestjs/core`, `@nestjs/common`, `@nestjs/platform-express`)
- **Configuration & Guards**: `@nestjs/config` (`v4.0.4`), `@nestjs/throttler` (`v6.5.0`)
- **Database ORM**: Prisma Client (`v7.8.0`), Prisma CLI (`v7.8.0`)
- **Database Engine**: PostgreSQL 16 (Hosted on Supabase Cloud / Docker Container)
- **Authentication & Security**: Passport JWT (`@nestjs/passport` `v11.0.5`, `@nestjs/jwt` `v11.0.2`, `passport-jwt` `v4.0.1`, `passport` `v0.7.0`), Bcrypt (`v6.0.0`)
- **Validation & Transformation**: `class-validator` (`v0.15.1`), `class-transformer` (`v0.5.1`), `validator` (`v13.15.35`)
- **File & Media Storage**: Cloudinary Node SDK (`v2.10.0`) via Express Multer memory storage

- **Payment Processing**: Currently handled via synchronous internal mock transaction handling (Stripe SDK is not installed in dependencies and no Stripe webhooks exist in compiled backend).

---

## 3. Architecture Overview

The system follows a decoupled client-server architecture with secure API boundary controls:

```
graph TD
    Client[Next.js 16 Frontend App<br/>Port 3000]

    subgraph NestJS Backend Services [NestJS 11 Backend A
        AuthModule[Auth Module<br/>JWT & Bcrypt]
        UsersModule[Users & Profile Module]
        ProductsModule[Products & Gatekeeper Module]
        AdminModule[Admin Moderation & Analytics]
        TxModule[Transactions Module]
        UploadsModule[Uploads Module<br/>Multer Memory St
    end

    Database[(Supabase PostgreSQL 16<br/>Prisma ORM v7.8)
    Cloudinary Service[Cloudinary Cloud Media CDN]

    Client -->|HTTP REST / Bearer JWT| AuthModule
    Client -->|HTTP REST / Bearer JWT| UsersModule
    Client -->|HTTP REST / Bearer JWT| ProductsModule
    Client -->|HTTP REST / Bearer JWT| AdminModule
    Client -->|HTTP REST / Bearer JWT| TxModule
    Client -->|Multipart FormData| UploadsModule

    AuthModule -->|Prisma Client| Database
    UsersModule -->|Prisma Client| Database
    ProductsModule -->|Prisma Client| Database
    AdminModule -->|Prisma Client| Database
```

```
    TxModule -->|Prisma Client| Database

    UploadsModule -->|Buffer Upload API| Cloudinary Servi
    ProductsModule -.->|Deliverable File Access Check| Tx
```

---

# 4. Database Schema

Extracted directly from `node_modules/.prisma/client/schema.prisma`:

## Enums

- **`Role`**: `user`, `admin`
- **`ProductStatus`**: `pending`, `approved`, `rejected`, `sold`

## Data Models

### 1. `User`

| Field | Type | Attributes / Constraints | Description |
|---|---|---|---|
| `id` | String | `@id @default(uuid())` | Unique user identifier |
| `email` | String | `@unique` | Account email address |
| `password_hash` | String | | Bcrypt hashed password |

| | | | |
|---|---|---|---|
| `role` | Role | `@default(user)` | User authorization role (`user` or `admin`) |
| `is_banned` | Boolean | `@default(false)` | Account ban status flag |
| `banned_reason` | String? | | Optional reason for account ban |
| `banned_at` | DateTime? | | Timestamp when account was banned |
| `created_at` | DateTime | `@default(now())` | Account creation timestamp |

### 2. `Profile`

| Field | Type | Attributes / Constraints | Descri |
|---|---|---|---|
| `id` | String | `@id @default(uuid())` | Profile identifi |
| `user_id` | String | `@unique`, FK -> `User.id` (Cascade) | User re |
| `username` | String | `@unique` | Public unique userna handle |
| `name` | String | | Display name |
| `bio` | String? | | Short biograp |

| | | | |
|---|---|---|---|
| `avatar_url` | String? | | Cloudi avatar URL |
| `is_verified` | Boolean | `@default(false)` | Studen verifica flag |
| `phone` | String? | | Contac phone numbe |
| `student_id` | String? | | Univers studen numbe |
| `student_id_card_url` | String? | | Upload studen card im URL |
| `verification_status` | String | `@default("unverified")` | State: `unver` `pendi` `verif` `rejec` |
| `verification_reason` | String? | | Reason verifica decisio |
| `notify_email` | Boolean | `@default(true)` | Email notifica toggle |
| `notify_push` | Boolean | `@default(true)` | Push notifica toggle |
| `notify_sms` | Boolean | `@default(false)` | SMS notifica toggle |

### 3. `Category`

| Field | Type | Attributes / Constraints | Description |
|---|---|---|---|
| `id` | String | `@id @default(uuid())` | Category identifier |
| `name` | String | `@unique` | Category name |

### 4. `ProductType`

| Field | Type | Attributes / Constraints | Description |
|---|---|---|---|
| `id` | String | `@id @default(uuid())` | Product type identifier |
| `category_id` | String | FK -> `Category.id` (Cascade) | Parent category relation |
| `name` | String | | Sub-category / product type name |
| Constraint | | `@@unique([category_id, name])` | Unique name per category |

### 5. `Product`

| Field | Type | Attributes / Constraints | Descr |
|---|---|---|---|
| `id` | String | `@id @default(uuid())` | Produc listing identifi |
| `user_id` | String | FK -> `User.id` (Cascade) | Seller relatio |

| | | | |
|---|---|---|---|
| `title` | String | | Listing |
| `description` | String | | Listing detaile descrip |
| `price` | Float | | Price i ETB |
| `status` | ProductStatus | `@default(pending)` | Moder status |
| `category_id` | String | FK -> `Category.id` | Associ catego |
| `product_type_id` | String | FK -> `ProductType.id` | Associ produc type |
| `is_featured` | Boolean | `@default(false)` | Featur placem flag |
| `is_hidden` | Boolean | `@default(false)` | Seller hidden toggle |
| `report_count` | Int | `@default(0)` | Count user fl reports |
| `deliverable_file_url` | String? | | Protec Cloudi file downlo URL |
| `created_at` | DateTime | `@default(now())` | Creatio timesta |

### 6. `ProductImage`

| Field | Type | Attributes / Constraints | Description |
|---|---|---|---|

| | | | |
|---|---|---|---|
| `id` | String | `@id @default(uuid())` | Image record identifier |
| `product_id` | String | FK -> `Product.id` (Cascade) | Parent product listing |
| `image_url` | String | | Cloudinary photo URL |

### 7. `ProductReview`

| Field | Type | Attributes / Constraints | Description |
|---|---|---|---|
| `id` | String | `@id @default(uuid())` | Admin review record identifier |
| `product_id` | String | FK -> `Product.id` (Cascade) | Moderated product listing |
| `admin_id` | String | FK -> `User.id` (Cascade) | Reviewing administrator |
| `status` | ProductStatus | | Moderation decision (`approved` / `rejected`) |
| `reason` | String? | | Mandatory reason if rejected |
| `reviewed_at` | DateTime | `@default(now())` | Timestamp of review |

### 8. `Transaction`

| Field | Type | Attributes / Constraints | Description |
|---|---|---|---|

| | | | |
|---|---|---|---|
| `id` | String | `@id @default(uuid())` | Transaction identifier |
| `product_id` | String | FK -> `Product.id` | Purchased product listing |
| `buyer_id` | String | FK -> `User.id` | Buyer user relation |
| `payment_id` | String? | | Payment intent ID (null for mock checkout) |
| `status` | String | `@default("pending")` | State (e.g. `completed`) |
| `is_test` | Boolean | `@default(true)` | Test transaction indicator |
| `buyer_comment` | String? | | Post-purchase buyer review comment |
| `buyer_rating` | Int? | | Post-purchase buyer rating (1-5) |
| `created_at` | DateTime | `@default(now())` | Purchase timestamp |

## Database Migration History

- No raw SQL migration scripts exist under a `prisma/migrations` directory in the repository.
- Schema changes were directly pushed and synchronized to the

live Supabase PostgreSQL database using Prisma CLI (`prisma db push`).

---

# 5. API Reference

Every endpoint listed below has been verified against backend controllers:

### Authentication Module (`/auth`)

| Method | Path | Auth Required | Description |
|---|---|---|---|
| `POST` | `/auth/register` | Public | Register new user account with profile |
| `POST` | `/auth/login` | Public | Authenticate credentials, returns access & refresh tokens |
| `POST` | `/auth/refresh` | Public | Generate new access token via refresh token |

### Users Module (`/users`)

| Method | Path | Auth Required | Description |
|---|---|---|---|
| `GET` | `/users/me` | JWT | Get current authenticated user profile |
| `GET` | `/users/me/notifications` | JWT | Fetch recent synthesized notifications (last 7 days reviews & sales) |

| | | | |
|---|---|---|---|
| `GET` | `/users/:id` | JWT | View public profile of a user by ID |
| `PATCH` | `/users/me` | JWT | Update authenticated profile details |
| `POST` | `/users/password` | JWT | Change password with current password validation |

## Products Module (`/products`)

| Method | Path | Auth Required | Description |
|---|---|---|---|
| `GET` | `/products` | Public | Query approved/sold listings (filterable by category, type, search, seller) |
| `GET` | `/products/mine` | JWT | Get listings created by the authenticated user |
| `GET` | `/products/:id` | Optional Bearer | Get single product detail (returns purchase state if token present) |
| `GET` | `/products/:id/download` | JWT | Securely fetch deliverable file URL (Owner, Admin, or Buyer only) |
| `POST` | `/products` | JWT | Create product listing (initial status: `pending`) |
| `PATCH` | `/products/:id` | JWT | Update listing details (resets status to `pending`) |

| | | | |
|---|---|---|---|
| `DELETE` | `/products/:id` | JWT | Delete product listing (owner only) |
| `PATCH` | `/products/:id/toggle-hide` | JWT | Toggle hidden state of owned listing |

### Categories Module (`/categories` & `/product-types`)

| Method | Path | Auth Required | Description |
|---|---|---|---|
| `GET` | `/categories` | Public | List all categories with product types & product counts |
| `GET` | `/product-types` | Public | List product types, optionally filtered by `category_id` |

### Transactions Module (`/transactions`)

| Method | Path | Auth Required | Description |
|---|---|---|---|
| `GET` | `/transactions` | JWT | List authenticated buyer's purchase history |
| `GET` | `/transactions/sales` | JWT | List authenticated seller's sales history |
| `GET` | `/transactions/:id` | JWT | Get transaction details by ID |
| `POST` | `/transactions` | JWT | Execute instant purchase for a listing (`product_id`) |

| | | | |
|---|---|---|---|
| `DELETE` | `/transactions/:id` | JWT | Delete purchase record from buyer history |
| `POST` | `/transactions/:id/review` | JWT | Submit buyer review comment and 1-5 rating |

## Uploads Module (`/uploads`)

| Method | Path | Auth Required | Description |
|---|---|---|---|
| `POST` | `/uploads/image` | JWT | Upload image file to Cloudinary (Max 5MB, image mimetypes) |
| `POST` | `/uploads/file` | JWT | Upload deliverable file to Cloudinary (Max 10MB) |

## Admin Moderation & Analytics Module (`/admin`)

| Method | Path | Auth Required | Description |
|---|---|---|---|
| `GET` | `/admin/stats` | Admin JWT | System metrics, revenue totals, SLA stats, category breakdown |
| `GET` | `/admin/stats/timeseries` | Admin JWT | Daily listings and transactions metrics (7d / 30d) |

| | | | |
|---|---|---|---|
| GET | /admin/activity/recent | Admin JWT | Recent system audit feed (listings, transactions, reports) |
| GET | /admin/products | Admin JWT | View pending listings review queue |
| GET | /admin/products/all | Admin JWT | View all non-pending listings (paginated, searchable) |
| PATCH | /admin/products/:id/review | Admin JWT | Approve listing or reject with mandatory reason |
| DELETE | /admin/products/:id | Admin JWT | Admin delete any listing |
| GET | /admin/users | Admin JWT | View paginated user directory (searchable, role filter) |
| GET | /admin/users/:id | Admin JWT | View user profile, listings, and transaction details |
| PATCH | /admin/users/:id/ban | Admin JWT | Ban user account with reason (blocks admins from being banned) |
| PATCH | /admin/users/:id/unban | Admin JWT | Unban user account |

| | | | |
|---|---|---|---|
| GET | /admin/transactions | Admin JWT | View global transaction ledger (paginated, searchable) |
| GET | /admin/verifications | Admin JWT | View student ID verification queue |
| PATCH | /admin/verifications/:userId | Admin JWT | Approve, reject, or revoke student verification status |
| GET | /admin/categories | Admin JWT | Manage category list |
| POST | /admin/categories | Admin JWT | Create category |
| PATCH | /admin/categories/:id | Admin JWT | Update category name |
| DELETE | /admin/categories/:id | Admin JWT | Delete category (blocked if products linked) |
| GET | /admin/product-types | Admin JWT | Manage product types |
| POST | /admin/product-types | Admin JWT | Create product type |
| PATCH | /admin/product-types/:id | Admin JWT | Update product type name |
| DELETE | /admin/product-types/:id | Admin JWT | Delete product type (blocked if products linked) |

# 6. Features Implemented

*All features listed below have been explicitly verified, tested, and confirmed working by the project user.*

## Standard User Features

- **User Authentication**: Complete register, login, and JWT refresh token flow.
- **Account & Profile Management**: Edit profile info (name, bio, phone, avatar), change password with current password verification.
- **Student ID Verification Submission**: Upload student ID card photo and enter student ID number.
- **Marketplace Discovery**: Search listings by keyword, filter by Category and Product Type, view listing detail pages.
- **Product Listing Creation**: Submit digital product with title, description, price, category, product type, images, and deliverable file.
- **Listing Management**: Edit owned listings (automatically resets status to `pending`), toggle hidden state, or delete listing.
- **Instant Purchase**: Execute purchase with self-purchase protection.
- **Deliverable File Gatekeeping**: Secure server-side download gate restricting access to owner, admin, or verified buyer.
- **Sales & Purchase History**: View personal purchases and sales history dashboards.
- **Post-Purchase Reviews**: Submit 1 to 5-star ratings and text comments on purchased resources.
- **In-App Notification Feed**: Auto-generated 7-day notification log for listing approvals/rejections and completed sales.

## Admin Features

- **Analytics Overview**: Real-time platform counts, revenue aggregation, category breakdown, review SLA metrics.
- **Time-Series Charts**: 7-day and 30-day interactive daily metrics.
- **System Activity Feed**: Live audit log of recent listings, sales, and flag reports.
- **Product Moderation Queue**: Approve pending listings or reject with mandatory feedback.
- **Global Listing Directory**: Full oversight of all listings with search, category filtering, and administrative deletion.
- **User Moderation & Banning**: Search directory, inspect profiles, unban, and ban users (with admin protection rule). Banning invalidates active JWT tokens on subsequent requests.
- **Student Verification Moderation**: Approve, reject, or revoke student ID verification submissions.
- **Category & Product Type Management**: CRUD operations with relational integrity protection.
- **Global Transaction Ledger**: Searchable audit log of all platform purchases.

---

# 6.5. System Behavior & Business Logic

## Standard User

### 1. Required Fields & Validation Rules for Product Listing Submission

Enforced server-side via NestJS `ValidationPipe` using `CreateProductDto`:

- **`title`**: Required string, maximum length 120 characters (`@IsString()`, `@IsNotEmpty()`, `@MaxLength(120)`).
- **`description`**: Required string, non-empty (`@IsString()`, `@IsNotEmpty()`).
- **`price`**: Required number, minimum value 1 ETB (`@IsNumber()`, `@Min(1)`, `@Type(() => Number)`).

- **`category_id`**: Required valid category UUID string (`@IsString()`, `@IsNotEmpty()`).
- **`product_type_id`**: Required valid product type UUID string (`@IsString()`, `@IsNotEmpty()`).
- **`images`**: Optional array of valid URL strings (`@IsOptional()`, `@IsArray()`, `@IsUrl({}, { each: true })`).
- **`deliverable_file_url`**: Optional string URL (`@IsOptional()`, `@IsString()`).

### 2. Listing Status Transitions

- **Creation (`POST /products`)**: Initial status automatically set to `pending`.
- **Seller Update (`PATCH /products/:id`)**: Any edit by seller automatically resets status back to `pending`.
- **Admin Review (`PATCH /admin/products/:id/review`)**: Admin sets action `approve` -> `approved`, or `reject` -> `rejected` (requires reason).
- **Hide Toggle (`PATCH /products/:id/toggle-hide`)**: Toggles `is_hidden` boolean property on product without changing `status`.

### 3. Verified Student Logic

- **Mechanics**: Purely manual admin review workflow.
- Standard users upload `student_id` and `student_id_card_url` on their profile.
- Verification status transitions from `"unverified"` -> `"pending"`.
- Admin inspects the image URL and approves or rejects it in `/admin/verifications`.
- There is no automated email domain regex check or OCR student card validation in code.

### 4. Self-Purchase Prevention Rule

- Enforced in `TransactionsService.create` (lines 66-68):

```
if (product.user_id === buyerId) {
    throw new BadRequestException('You cannot purchase yo
}
```

#### 5. Full Purchase Flow

1. Buyer clicks **"Buy Now"** on frontend product page.
2. Frontend sends `POST /transactions` with body `{ product_id }` and Bearer JWT token.
3. Backend validates JWT token, checks product existence, and ensures seller !== buyer.
4. Backend creates a `Transaction` row synchronously with `payment_id: null`, `status: 'completed'`, `is_test: true`.
5. Response returns completed transaction object.
6. When buyer clicks **"Download File"**, frontend calls `GET /products/:id/download`.
7. Backend verifies buyer ID has a `status: 'completed'` transaction, then returns `{ deliverable_file_url }`.

#### 6. Information Access: Before vs After Purchase

- **Before Purchase**: Buyer views listing title, description, price, category, product type, seller info, and images. The `deliverable_file_url` field is explicitly removed (`delete product.deliverable_file_url`) in `findAll` and `findOne` responses.
- **After Purchase**: `GET /products/:id` returns `has_purchased: true` and `purchased_at`. Buyer gains authorization to call `GET /products/:id/download` to retrieve `deliverable_file_url`, and can submit ratings/reviews via `POST /transactions/:id/review`.

---

## Admin

**1. Listing Review Criteria**

- No automated quality rubric, keyword filter, or automated AI scanner exists in code.
- Approval/rejection is based purely on manual admin judgment.

**2. Seller Rejection Notification**

- When an admin rejects a product, a `ProductReview` row is stored with `status: 'rejected'` and `reason`.
- When the seller visits their dashboard, `UsersService.getNotifications` queries `ProductReview` records for products owned by that user from the last 7 days and synthesizes a notification object (`type: 'rejection'`, product title, rejection reason).

**3. User Banning Mechanics**

- `AdminService.banUser` sets `is_banned: true`, `banned_reason`, and `banned_at` on the `User` database record.
- Every API request authenticated via `JwtStrategy.validate` executes a database lookup:

```
if (user.is_banned) {
    throw new ForbiddenException('Your account has been s
}
```

- **Effect**: Banning a user blocks future logins AND **immediately invalidates all active, previously-issued JWT tokens** on their next HTTP request.

**4. Safeguard Against Admin Banning**

- Enforced in `AdminService.banUser` (lines 332-334):

```
if (user.role === 'admin') {
    throw new BadRequestException('Cannot ban an admin ac
```

```
}
```

- Admins cannot ban other admin accounts.

#### 5. Admin Student Verification Flow

- Admin calls `GET /admin/verifications` which returns all profile records.
- Admin calls `PATCH /admin/verifications/:userId` with `{ action: 'approve' | 'reject' | 'revoke', reason?: string }`.
- `approve` sets `is_verified: true`, `verification_status: 'verified'`, `verification_reason: null`.
- `reject`/`revoke` sets `is_verified: false`, `verification_status: 'rejected'`, `verification_reason: reason`.

---

## 7. Known Limitations / Not Yet Implemented

1. **Stripe Payment Gateway**: Stripe SDK is not installed, and no Stripe webhook handlers are implemented. Transactions complete instantly via internal mock workflow.
2. **Automated Testing**: Jest configuration exists, but unit and end-to-end test suites are unwritten.
3. **CI/CD Pipelines**: No GitHub Actions or automated deployment pipelines are currently configured.
4. **Production Build Setup**: Frontend and backend run in development environments (`next dev` and local NestJS dev server). Production build scripts have not been executed or deployed to production hosts.
5. **Deferred Features**: Direct real-time messaging between users and product review comments from non-buyers are explicitly deferred.

---

# 8. Environment Setup Instructions

## Prerequisites

- Node.js (`v20.x` or higher)
- npm (`v10.x` or higher)
- Docker Desktop (Optional, for local PostgreSQL)

## 1. Database & Local Environment Configuration

Navigate to `backend/` and verify `.env` file containing required parameters:

```
DATABASE_URL="postgresql://postgres:postgres@localhost:54
JWT_SECRET="your_jwt_access_secret_key"
JWT_REFRESH_SECRET="your_jwt_refresh_secret_key"
JWT_ACCESS_EXPIRES_IN="15m"
JWT_REFRESH_EXPIRES_IN="7d"
PORT=3002

CLOUDINARY_CLOUD_NAME="your_cloudinary_cloud_name"
CLOUDINARY_API_KEY="your_cloudinary_api_key"
CLOUDINARY_API_SECRET="your_cloudinary_api_secret"
```

## 2. Start PostgreSQL via Docker (If running database locally)

Run from root directory:

```
docker-compose up -d db
```

## 3. Setup and Run Backend API

```
cd backend
npm install
npx prisma db push
npm run start:dev
```

*Backend API will start on `http://localhost:3002`.*

### 4. Setup and Run Frontend Application

From project root:

```
npm install
npm run dev
```

*Frontend App will start on `http://localhost:3000`.*

---

# 9. Deployment Info

- **Current Database Hosting**: Managed PostgreSQL Cloud instance on Supabase.
- **Current Media Storage**: Cloudinary Cloud CDN.
- **Local Application Ports**: Frontend (`:3000`), Backend API (`:3002`).

### Next Steps for Production Deployment

1. Build frontend production assets (`npm run build`).
2. Deploy Next.js frontend to Vercel or Node.js server.
3. Deploy NestJS backend API to container hosting (Render, AWS ECS, or DigitalOcean App Platform).
4. Configure production domain, SSL certificates, CORS origin restrictors, and production environment secrets.

---

# 10. Third-Party Services

1. **Cloudinary**: Active in free/sandbox tier for image uploads and deliverable file storage.
    - *Production transition*: Upgrade to paid tier if file bandwidth or storage exceeds free quota.
2. **Stripe**: Currently inactive (Stripe SDK and webhook integration deferred).
    - *Production transition*: Install `@stripe/stripe-js` & `stripe`, implement payment intent endpoints, and replace internal mock checkout logic with webhook verification.